

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511025
Name	S. Swedha

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 10%

QUESTIONS:2

Total Marks:20

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I __S. Swedha/192511025__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Answers:

1)

A smart home automation system manages lighting, temperature, and security by responding to real-time inputs motion sensors, temperature readings, time of day while also learning user preferences over time. This kind of system is best understood through the four classical types of intelligent agents: simple reflex, model-based reflex, goal-based, and utility-based agents. Each represents a different level of reasoning, and analyzing all four shows that a real smart home actually needs a combination of them working together, not just one in isolation

Simple Reflex Agent

A simple reflex agent acts only on the current percept, using fixed condition-action rules, with no memory of the past and no consideration of future consequences

In the smart home: "IF motion detected in living room, THEN turn on light" or "IF temperature > 28°C, THEN turn on AC." The agent reacts instantly but has no idea what happened a moment ago.

This works for very simple, immediate tasks, but breaks down quickly if a person briefly steps out of the motion sensor's range, the agent switches the light off immediately, even though the person is still in the room, since it has no memory of the room being occupied a second earlier.

Model-Based Reflex Agent

A model-based agent maintains an internal model of the world, allowing it to handle situations where the current percept alone isn't enough.

In the smart home, it would track things like "room occupied = true" even when the motion sensor briefly stops firing, only marking it "unoccupied" after a timeout with no motion. It could also track "it is currently night" or "security is armed," combining this remembered state with new sensor input — for example, avoiding bright lights at 3 AM even if motion is detected, since its internal

model flags this as unusual and possibly a security concern rather than a normal wake-up.

This avoids the flickering and false reactions of the simple reflex agent, because decisions are based on remembered context, not just the instantaneous sensor reading.

Goal-Based Agent

A goal-based agent has an explicit goal it is trying to reach, and chooses actions expected to lead toward that goal, reasoning over multiple steps if needed

In the smart home, a goal might be: "bedroom at 22°C by 10 PM." The agent reasons backward — how long does the AC take to cool the room, and when should it start — to satisfy this goal. Similarly, "all doors locked before the house is marked empty" requires checking and acting on a sequence of conditions, not just one sensor reading.

This gives the system the ability to plan ahead, but requires a defined goal state and some reasoning process to work out how to reach it.

Utility-Based Agent

A utility-based agent goes further — it uses a utility function to weigh trade-offs between competing objectives and picks the action that maximizes overall satisfaction, not just whichever action satisfies a goal.

This matters because comfort, energy efficiency, and security often conflict. Keeping the house perfectly comfortable all day wastes energy if no one is home; keeping every light on all night is more secure but costlier. A utility-based agent assigns scores to different action combinations — e.g., "cool the house to 24°C only from 6 PM," balancing comfort against energy cost — effectively answering "how well" something is achieved, not just whether it is achieved at all.

Algorithm: Hybrid Agent Decision Cycle

Since the most effective design is a hybrid of all four agent types (justified below), the actual decision-making of the system can be expressed as a single decision-cycle algorithm the agent runs continuously:

1. Sense current percepts:
 - motion status, temperature reading, time of day, door/window sensor states
2. Update internal model (model-based layer):
 - update occupancy status (with timeout logic, not just raw motion reading)
 - update "day/night" state
 - update "armed/disarmed" security state
3. Check for safety-critical conditions (simple reflex layer):
 - IF intrusion sensor triggered → trigger alarm immediately and skip remaining steps (no deliberation delay allowed)
 - IF smoke/gas sensor triggered → trigger alarm immediately
4. Check active goals (goal-based layer):
 - FOR each scheduled goal (e.g., "22°C by 10 PM", "Doors locked by away-mode"):
 - compute remaining time and remaining action needed
 - IF action must start now to meet the goal on time → queue that action
5. Resolve trade-offs among queued actions (utility-based layer):
 - FOR each candidate action (or combination of actions):
 - compute utility = weighted sum of (comfort score, energy-cost score, security score)

- select the action (or action set) with the highest overall utility

6. Execute selected action(s):

- send commands to actuators (lights, HVAC, locks)

7. Learning update:

- compare predicted user satisfaction/behavior with actual observed behavior (e.g., user manually overrides a setting)
- adjust rule thresholds, goal timing, or utility weights accordingly

8. Repeat from Step 1 (continuous loop)

Justification: Most Effective Approach

No single agent type on its own is sufficient. A simple reflex agent alone causes erratic, short-sighted behavior (flickering lights, no context awareness, and it cannot plan ahead or weigh trade-offs at all). This system genuinely needs the layered hybrid described in the algorithm above:

- Simple reflex behavior for safety-critical, instant responses (e.g., alarm triggers) where there is no time for deliberation.
- A model-based layer underneath everything, tracking occupancy, time, and current state, since nearly every smarter decision depends on this remembered context.
- Goal-based reasoning for scheduled tasks like pre-cooling a room or securing the house before "away mode."
- A utility-based layer to resolve the real conflicts between comfort, energy savings, and security.

- A learning mechanism wrapped around all of this, personalizing the rules, goals, and utility weights to the specific household over time.

2)

A robot placed in an unknown maze must find the exit with no prior map of the layout. Since it has no heuristic information about which direction is "closer" to the goal, this is a natural fit for uninformed (blind) search strategies — specifically Uniform Cost Search (UCS) and Iterative Deepening Search (IDS), both of which work without any domain-specific knowledge of where the goal lies.

How Uniform Cost Search (UCS) Applies

UCS explores the maze by always expanding the node (cell) with the lowest cumulative path cost $g(n)$ found so far, using a priority queue instead of a simple FIFO/LIFO structure. If every move costs the same, UCS behaves like Breadth-First Search, exploring the maze outward in expanding rings from the start. If some paths cost more (e.g., a narrow or difficult passage costs more than an open corridor), UCS becomes essential, because it guarantees the robot finds the truly cheapest path, not just the path with the fewest steps.

Applied to the maze: the robot starts at its position, treats each adjacent unblocked cell as a possible move, and always continues from the cheapest accumulated-cost cell in its frontier, until the cell it pops from the frontier is the exit.

Algorithm: Uniform Cost Search

1. Initialize a priority queue (frontier) with the Start node,
 $g(\text{Start}) = 0$
2. Initialize an empty explored set
3. Loop while the frontier is not empty:
 - a. Remove the node with the lowest $g(n)$ from the frontier

- b. IF this node is the Goal $\rightarrow$ return the path (success)
- c. Add this node to the explored set
- d. FOR each neighbor reachable from this node (Up, Down, Left, Right, if not blocked):
 - i. IF neighbor is in explored set $\rightarrow$ skip
 - ii. Compute tentative $g(\text{neighbor}) = g(\text{current}) + \text{cost}(\text{move})$
 - iii. IF neighbor is not in frontier, OR the new g is lower than its existing value in the frontier:
 - update $g(\text{neighbor})$
 - set $\text{parent}(\text{neighbor}) = \text{current}$
 - add/update neighbor in the frontier
- 4. IF frontier becomes empty $\rightarrow$ return failure (no exit reachable)

How Iterative Deepening Search (IDS) Applies

IDS combines the space-efficiency of Depth-First Search with the completeness of Breadth-First Search. It performs a depth-limited DFS with a small depth limit first, and if the exit isn't found, discards everything and repeats with the limit increased by one, continuing until the exit is found.

Applied to the maze: the robot first tries all paths of length 1, then restarts and tries all paths up to length 2, then up to length 3, and so on — never wandering arbitrarily deep down one corridor while ignoring a nearby exit, and only needing to remember the current path, not every visited cell.

Algorithm: Iterative Deepening Search

1. FOR $\text{depth_limit} = 0, 1, 2, 3, \dots$ (increasing indefinitely):
 - a. $\text{result} = \text{Depth-Limited-Search}(\text{Start}, \text{depth_limit})$
 - b. IF $\text{result} = \text{success}$ $\rightarrow$ return the path found
 - c. IF $\text{result} = \text{"cutoff never reached"}$ (i.e., the whole maze was exhausted at this depth without finding the exit) $\rightarrow$ return failure (no exit exists)

d. Otherwise, continue to the next depth_limit

Depth-Limited-Search(node, limit):

1. IF node is the Goal $\rightarrow$ return success (path found)
2. IF limit = 0 $\rightarrow$ return cutoff (stop going deeper here)
3. FOR each neighbor of node (Up, Down, Left, Right, if not blocked, and not already on the current path):
 - a. result = Depth-Limited-Search(neighbor, limit - 1)
 - b. IF result = success $\rightarrow$ return success
4. Return cutoff (or failure if no neighbors existed at all)

Advantages and Disadvantages

Uniform Cost Search:

Advantage — guaranteed to find the optimal (least-cost) path, which matters if some maze passages are costlier to traverse than others. It is complete: it will always find an exit if one exists and costs are non-negative.

Disadvantage — space complexity can be very high, since it must store every node in the frontier along with its accumulated cost, growing quickly (exponentially in the worst case) as the maze gets larger. Time complexity is similarly exponential in the worst case.

Iterative Deepening Search:

Advantage — uses very little memory (space complexity proportional only to the current search depth, not the whole maze), since at each iteration it behaves like DFS and never stores the whole frontier. It is complete for a finite maze, and works even when the depth of the goal is unknown in advance.

Disadvantage — repeatedly re-explores the shallow parts of the maze at every iteration, wasting computation. It also does not guarantee the cheapest path if move costs vary — it only guarantees the path with the fewest moves, since it doesn't track cumulative cost the way UCS does.

Relating the Problem to Intelligent Agent Types

The maze-solving robot is best understood as a goal-based agent at minimum, since its entire behavior is driven by reaching a defined goal state (the exit), requiring reasoning over a sequence of actions rather than simply reacting to the immediate percept the way a simple reflex agent would.

However, because the maze is unknown, the robot also needs a model-based component: as it explores, it must build and update an internal map of which cells it has visited and which paths are dead ends, so it doesn't repeat the same mistake twice. A pure simple reflex agent (no memory) would be nearly useless here, since it could walk in circles forever, repeatedly re-entering the same dead end with no memory of having tried it before.

If the maze had costs that varied significantly between paths (e.g., rough terrain, energy cost of movement), the robot would additionally benefit from utility-based reasoning, weighing which of several valid paths is most efficient overall which is effectively what UCS computes through its cost function.

This shows how agent design directly affects decision-making: a simple reflex design is incapable of solving an unknown maze at all; a model-based design is the minimum requirement to avoid repeating failed paths; and a goal-based (or utility-based, if costs vary) design is what actually drives the systematic search toward the exit.

Suggested Combination and Justification

The most effective combination for this problem is a model-based, goal-based agent using Uniform Cost Search, when memory is not severely constrained and path costs vary across the maze. This is because:

- The model-based component is essential simply to make search possible at all in an unknown environment — without memory of visited cells, the robot cannot systematically explore.

- The goal-based component gives the robot a clear termination condition (reaching the exit) and the reasoning structure needed to evaluate sequences of moves rather than single reactions.
- UCS is preferred over IDS specifically because it guarantees the truly cheapest path to the exit, which matters when some routes are significantly more costly (slower, riskier, or longer) than others, not just different in step count.

If, instead, the robot has strict memory limitations (e.g., a lightweight embedded system) and all moves in the maze have equal cost, a model-based, goal-based agent using IDS becomes the better choice, since it achieves the same completeness guarantee using far less memory, and the equal-cost assumption means IDS's step-count-based result is automatically also cost-optimal.